TrustPro A Trust-Based Local Services & Skilled-Labour Marketplace for Cameroon

Software Requirements Specification & System Architecture Consolidated, build-ready specification (IEEE 830-inspired) Version 3.0 · July 2026 Pillars: Trust · Discovery · Transactions · Progress Contents

PART I Software Requirements Specification 1. Introduction 1.1 Purpose This document specifies the functional and non-functional requirements and the target architecture for TrustPro, a trust-first marketplace that connects Cameroonian households and businesses with verified skilled workers. It consolidates four inputs into one coherent, non-duplicated specification: the team's internal Trust Engine and progressive-verification analysis, the technical stack decision record (the "2026 Universal Stack"), the functional decisions from the project team meeting, and a 27-respondent user survey ("Shaping the Future of Local Services in Cameroon"). It is written to be executed directly with an AI-assisted development workflow and is the single source of truth for product, engineering, and QA.

1.2 Product scope TrustPro is a two-sided marketplace for the Cameroonian informal and skilled-labour market (electricians, plumbers, welders, mechanics, painters, carpenters, cleaners, and similar trades). It is delivered as two distinct but connected experiences over one platform: • Marketing website, a conversion-optimised, SEO-facing public site (value proposition, how-it-works, safety, categories, testimonials, CTA) that is separate from the installable app shell and requires no authentication. • Application, an installable Progressive Web App (discovery, role-based dashboards, verification, messaging, escrow payments, trust and safety). WEB-ONLY: there is no native mobile app; the PWA installs on phones and desktops and "mobile" always means the responsive PWA in a phone browser. Delivery phasing (reconciled) The platform is delivered as a Web PWA only. The API remains deliberately client-agnostic so a native app could be added someday without rearchitecting, but no native app is planned or specified in this document. The marketing landing page stays outside the service-worker scope so it remains a fast, crawlable page. The competitive advantage is trust, not the listing. TrustPro is not Fiverr, Upwork, or TaskRabbit; the trust layer (a continuously updated Trust Engine, verified identity, portfolio evidence, transparent pricing, and escrow) is the product. Every requirement serves one of four pillars: Trust, Discovery, Transactions, Progress. 1.3 Market rationale (survey evidence) A 27-respondent survey validated which signals most influence adoption; each respondent selected exactly three features. The counts drive prioritisation: the four strongest are treated as primary pillars and Must-Have (Part III, §22), while live tracking is validated as lower priority and deferred. Feature (survey wording) Resp. % of 27 Product consequence Real photos of past projects (portfolio) 16 59.3% Portfolio is first-class evidence Verified ID / Trust Score for workers 16 59.3% Progressive verification + visible Trust Score Clear, fixed-price quotes before work starts 14 51.9% Transparent pricing and quotes up front Secure in-app payment (escrow) 12 44.4% Escrow held until completion (Campay) Emergency / SOS button 8 29.6% Safety treated as a trust baseline, not by vote share Live tracking of worker arrival 4 14.8% Deferred (Could-Have) 1.4 Definitions, acronyms, and abbreviations Term Meaning Guest An unauthenticated visitor who may browse but not act. Worker A user offering a trade/service (the supply side). Client / Worker-Finder A user hiring a worker or posting a job (the demand side). Business A registered employer (hotel, hospital, NGO, school, construction firm) hiring at team scale. Field Agent Internal role that recruits and personally verifies workers and inspects businesses. Admin Platform operator managing verification, fraud, payments, disputes, and configuration. Trust Engine The continuously updated scoring/tiering system (identity, professional, marketplace, financial). Trust Score A 0–100 composite reflecting verified identity, professional history, and marketplace behaviour. Tier A discrete verification level (Tier 0–Tier 3) that gates platform capabilities. Escrow Funds held by the platform until a job is confirmed complete. Role switch A one-tap control letting one account act as Worker and Client (and own a Business). Campay Payment aggregator for Mobile Money (MTN, Orange), card, and bank in XAF. KYC / OTP / RCCM Know-Your-Customer identity check / one-time password (delivered via email/console at launch; SMS later behind the channel service) / Cameroon business-registry number. FCFA / XAF Central African CFA franc, the platform currency. PWA / MVT Progressive Web App / Mapbox Vector Tiles (generated dynamically by CARTO). TMDS Trust Motion Design System, the product's design language (Part III). 1.5 References • Internal Trust Engine / progressive-verification architecture analysis (this project). • Team technical stack decision record, the "2026 Universal Stack" (this project). • Project team meeting notes on core platform functionality (this project). • User validation survey "Shaping the Future of Local Services in Cameroon" (n = 27). • Product design brief (TMDS philosophy and screen inventory); interface exploration set (ten batches). • Engineering standards, the battle-tested backend/frontend/SEO/observability lessons (applied in Part II). • IEEE 830, structural inspiration for this specification. 2. Overall Description 2.1 Product perspective TrustPro is a new, independently built system. Its core differentiator, per the team's internal analysis, is not identity verification alone but a continuously updated Trust Engine: a scoring and tiering system that combines identity, professional reputation, marketplace behaviour, and financial reliability, and that rewards workers for increasing their verification level rather than gatekeeping them from entry. It is built as a monorepo spanning a Next.js web app and a NestJS API over PostgreSQL/PostGIS, integrating Campay (payments), OSM/CARTO/Mapbox/BigDataCloud (geospatial), and AWS S3 (media). The location model is layered and human-augmented (Part II, §12). 2.2 Product vision and the trust journey Every screen reinforces one journey; the UI communicates safety before the user reads any text. Discover → Verify trust → Hire → Secure payment → Complete job → Build reputation 2.3 The four product pillars Pillar What it covers Trust Trust Engine (score, tiers), identity & document verification, portfolio evidence, reviews, endorsements, badges, safety/SOS. Discovery Guest-browsable search, categories, filters, nearby workers, recommendations, maps, availability, portfolio. Transactions Quotes, bookings, escrow, wallet, payments, payouts, invoices, disputes, job lifecycle. Progress Onboarding, profile completion, verification tiers, achievements, milestones, statistics, growth. 2.4 User classes and characteristics Six classes. Guests browse without friction; Workers and Clients/Businesses are the customers and share one account via role switching; Field Agents and Admins are operational. Most users are non-technical, mobile-first, and price-sensitive on data, which drives the PWA/offline and low-friction requirements. 2.4.1 Guest An unauthenticated visitor. Can browse worker profiles and posted jobs and use search/filter without an account; any state-changing action prompts registration and then resumes the original intent. 2.4.2 Worker • Goals: fast low-friction onboarding, get discovered, build trust, win jobs, get paid securely, raise Trust Score, reach Tier 2/3. • Key surfaces: dashboard, discovery, job requests, applications, quotes, jobs, availability, portfolio, services, reviews, Trust Score, documents, analytics, wallet, profile. 2.4.3 Client (household / individual / family) • Goals: confidence in worker legitimacy and quality, fair pricing, payment protection, safe hiring, tracking, reviews. • Key surfaces: dashboard, discover workers, saved workers, post-job, jobs, quotes, bookings, payments, messages, profile. 2.4.4 Business (verified employer) • Adds to Client: team/bulk hiring, multiple concurrent jobs, company verification (RCCM), team management with access levels, company wallet, business analytics, staff invitations, departments. • Key surfaces: business dashboard, team, jobs, applicants, departments, analytics, payments, company profile, verification. 2.4.5 Field Agent (operations) • Responsibilities: recruit workers, verify identities in person, inspect businesses, upload documents, review portfolios, approve onboarding, seed "Founding Professional" supply. • Key surfaces: agent dashboard, assigned verifications, pipeline, nearby workers, business inspections, coverage area & routes, pending reviews, reports, verification map. 2.4.6 Platform Admin • Responsibilities: configure service categories, review verification submissions, adjust Trust Scores in exceptional cases, resolve disputes, oversee payments/escrow, fraud, analytics, reports, announcements, settings. • Key surfaces: admin dashboard, users/workers/businesses/agents, verification queue, agents invite, trust management, fraud & risk, disputes, payments & escrow, categories, analytics, heat maps, audit logs, settings, RBAC. 2.5 Operating environment • Clients: installable Web PWA usable in any modern browser and on low/mid-range Android phone browsers over 3G/4G; installable on desktop too. Must degrade gracefully on intermittent connectivity. • Region: Cameroon, starting with cities Douala and Yaoundé. • Languages: English and French (full bilingual UI), reflecting Cameroon's dual official-language context. • Currency: FCFA / XAF only. • Payments: Mobile Money (MTN, Orange), card, and bank via Campay. 2.6 Design and implementation constraints • Guests browse the discovery interface freely; every state-changing action (post/apply/contact/pay/save/review) requires login, then returns the user to their original intent. • The two experiences differ deliberately: the marketing site is emotional and animated; the app is productive and calm. • Reads use GraphQL; writes use REST (Part II, §10). Spatial computation lives in PostGIS/CARTO, never on the client. • Search uses fuzzy matching and debounced input, scoped to an admin-configured category taxonomy, with a friendly "no provider yet in your area" fallback rather than an empty result. • Verification uses soft restrictions rather than hard blocks wherever safe (e.g. a missing ID limits job volume rather than blocking the account); job posting is gated at Tier 2+. • The design system (TMDS) is mandatory: all colour, spacing, radius, type, motion, and component

behaviour derive from tokens, with no hardcoded values. • Baseline engineering quality is non-negotiable: OWASP Top 10 mitigations, a clean high/critical scan status (e.g. Snyk) in CI/CD, full responsive design, and robust profile management. 2.7 Assumptions and dependencies • A final brand name and visual identity will be supplied; this document uses TrustPro as the standard token. • Campay is available and supports collection and payout in XAF for the covered operators. • OSM coverage and CARTO/Mapbox rendering suffice for the target cities; BigDataCloud serves reverse-geocoding. • Field agents exist operationally to perform in-person verification and inspection. • Users can receive SMS/OTP on Cameroonian mobile numbers(to come later, primary notifications are via push notifications and emails since SMS requires buying APIs); S3 (or compatible) is provisioned for media, documents, and voice notes. 3. System Features and Functional Requirements Requirements are grouped by module and tagged with stable IDs (module prefix + number) for traceability into UML, tests, and implementation prompts. “Shall” denotes a mandatory requirement. Each module maps to one or more pillars: Trust (T), Discovery (D), Transactions (X), Progress (P). Original-SRS FR numbers are cross-referenced where a requirement derives directly from that document. Prefix Module Pillars ACC Access Model, Accounts, Auth, Onboarding, Roles P, T DIS Discovery, Search & Location D PRO Profiles & Portfolio T, D TRU Trust Engine (Score, Verification, Tiers, Endorsements) T, P JOB Jobs & Applications (verification-gated) X QTE Quotes & Negotiation X MSG Messaging X, T PAY Payments, Escrow, Wallet, Payouts X, T REV Reviews & Ratings T SOS Safety & Emergency T NOT Notifications n/a BIZ Business & Team X, P AGT Agent Operations T ADM Admin Operations n/a SET Settings & Account Management n/a SYS Platform, PWA, i18n, System States n/a 3.1 Access Model, Accounts, Authentication, Onboarding & Roles (ACC) ID Requirement ACC-01 The system shall let Guests (unauthenticated) view the discovery interface (browse worker profiles and posted jobs and use search/filter) without creating an account. (SRS FR-4) ACC-02 Every state-changing action (post a job, apply, contact a worker, initiate payment, save a favourite, submit a review) shall require login; Guests attempting one are routed to registration/login and then returned to their original intent (e.g. resume the drafted job). (SRS FR-5) ACC-03 The public Marketing Landing Page shall be separate from the installable app shell, require no auth, and route users into the app via a clear CTA (“Find a Worker” / “Find Work”). (SRS FR-1, FR-2) ACC-04 Sign-up shall capture role intent up front (“Find Work” / “Hire Workers”) and verify the phone number via a 6-digit OTP with resend and countdown; email verification where supplied. ACC-05 The system shall issue stateless JWT sessions (Auth.js) that persist across devices, with refresh and secure storage; auth and OTP endpoints are rate-limited. ACC-06 One account shall hold multiple roles (Worker and Client), with Business ownership as an additional role; verification tier and Trust Score attach to the underlying identity, not per role, so switching never requires re-verification. (SRS FR-9, FR-11) ACC-07 A clearly visible, one-tap Role Switcher shall sit in primary navigation and swap the active experience and available actions without re-authentication. (SRS FR-10) ACC-08 The system shall provide login, logout, forgot/reset password, and optional two-factor authentication. ACC-09 Authentication errors shall be accurate: “Invalid phone/email or password” for bad credentials, never “Session expired” on a login failure; “Session expired” only on genuine expiry. ACC-10 Onboarding shall be role-specific: Worker (services, pricing, availability, portfolio, ID, selfie, location), Client (basic profile, location), Business (company details, documents, team), presented as a gamified completion flow (see TRU-05). ACC-11 The system shall support language selection (English/French) at first run and switching at any time, and shall render auth/onboarding inside the standard shell (split-screen on desktop), never “naked”. 3.2 Discovery, Search & Location (DIS) ID Requirement DIS-01 The home/discovery screen shall present, on one screen: a worker-profiles section, a posted-jobs section, a location-setter, a search bar, and a “Post a Job” CTA. (SRS FR-6) DIS-02 Worker cards shall surface at minimum: name/photo, profession, Trust Score/tier badge, sample portfolio photos, and starting/typical pricing; job cards shall surface title, category, location/distance, and any fixed-price range. (SRS FR-7, FR-8) DIS-03 The feed shall additionally include featured/top-rated workers, jobs nearby, recommended workers, trending categories, and a map preview. DIS-04 Search shall support fuzzy matching (tolerating typos/partial matches) and shall debounce input to avoid excessive queries while typing. (SRS FR-15) DIS-05 Search shall be scoped to an admin-configured taxonomy of service categories (see ADM-02), not free-text-only, so results stay meaningful and moderated. (SRS FR-16) DIS-06 If no worker in the area offers the requested service, the system shall show a friendly explicit message (e.g. “No providers for this service yet in your area, check back soon”) rather than an empty or ambiguous result. (SRS FR-17) DIS-07 Filters shall include location/proximity radius, service category, price range, minimum Trust Score/tier, rating, availability, verified-only, and response time; the active filter count shall be shown on the apply control. (SRS FR-18) DIS-08 Results shall be sortable/rankable, with higher-tier / higher-Trust-Score workers eligible for priority placement (see TRU-02, TRU-09). (SRS FR-19) DIS-09 The location-setter shall offer (a) one-tap “Use my current location” via device geolocation and (b) an interactive map to drop/adjust a pin; a manual pin or landmark note always overrides auto-detected location, and the set location drives both feed and search. (SRS FR-12–FR-14) DIS-10 The system shall provide a map/list split with clustered markers, nearby-worker previews, and a “search this area” control; saved/bookmarked workers, jobs, and searches, plus recently viewed, shall be supported. DIS-11 Long lists shall be virtualised and paginated; every list shall have skeleton loading and branded empty states. 3.3 Profiles & Portfolio (PRO) ID Requirement PRO-01 The public Worker Profile shall show cover, photo, animated Trust Score ring, tier badge, headline stats (jobs completed, rating, on-time %, verification), and tabs: About, Services, Portfolio, Reviews. PRO-02 Portfolio shall be first-class evidence (the top survey-validated driver): a gallery of photos/videos, project groupings, and before/after comparisons, with an “add more” flow for owners. (SRS FR-24) PRO-03 Workers shall offer structured fixed-price quoting for common job types in their category so clients see price expectations before contacting them; profiles display completed-job count, rating, and Trust Score/tier prominently. (SRS FR-25, FR-26) PRO-04 Profile actions: Message, Request a Quote / Book, Share, Report. PRO-05 Workers shall manage a Services list (name, category, description, starting price, active/inactive) shown as transparent pricing, and weekly Availability (per-day slots, exceptions, calendar) visible to clients in their area. PRO-06 Businesses shall have a verified Company Profile (logo, description, contact, RCCM/registration, member-since, aggregate stats) with tabs: About, Jobs, Reviews, Team, Documents. PRO-07 Owners shall edit profiles with inline success/error feedback; media uploads go to object storage with progress and validation. 3.4 Trust Engine, Score, Verification, Tiers, Endorsements (TRU) Core differentiator The Trust Engine is the platform’s core differentiator: a continuously updated score and tier system, deliberately framed as progress and achievement, never a one-time KYC checklist. Verification uses soft restrictions rather than hard blocks wherever safe, to maximise legitimate onboarding while still incentivising verification. (SRS §4.9, FR-28) ID Requirement TRU-01 The system shall compute a Trust Score (0–100) composed of four sub-scores (Identity, Professional, Marketplace, Financial) and display it as an animated ring wherever the entity appears; the full scoring table is in Part III, §21.5. (SRS FR-32) TRU-02 The system shall assign progressive Tiers (Tier 0–Tier 3) derived from the Trust Score and completed checks, with a visible tier badge; the exact tier gates and unlocks are in Part III, §21.4. (SRS §4.9.1) TRU-03 Unverified (Tier 0) profiles shall be explicitly labelled “Basic Profile”; the system shall never imply full verification for an unverified profile. (SRS FR-27) TRU-04 The system shall present a Trust Score breakdown screen explaining each sub-score, its inputs, and how to improve it, and factor the score into default search ranking. (SRS FR-34) TRU-05 Profile completion shall be shown as a progress bar with unlockable benefits (e.g. “Complete your profile, 60%. Unlock: Verified badge, more customers, priority search”) rather than a raw checklist. (SRS FR-29) TRU-06 The system shall run progressive checks (phone OTP, ID upload, ID approval, address, selfie/liveness, portfolio, field verification, jobs-completed and rating thresholds, dispute-free status), each with pending/verified/rejected states, timestamps, and re-upload on rejection. TRU-07 If automated ID verification (OCR) fails, the system shall show an actionable, non-judgemental retry message (e.g. “We couldn’t read your ID clearly. Try again in better lighting”). (SRS FR-30) TRU-08 The system shall manage a Documents area (National ID front/back, selfie, proof of address, certificates, portfolio images) with upload, status, and visibility controls; confirmed fraudulent documents shall trigger permanent suspension, overriding accumulated Trust Score. (SRS FR-33) TRU-09 The system shall support an internal “Founding Professional” path: an Admin/Field Agent can personally verify a worker recruited in person and grant an immediate “Founding Verified Professional” badge to seed trusted supply. (SRS FR-31) TRU-10 The system shall support community Endorsements (previous customer, workshop owner, quarter chief, landlord, community leader) that contribute a modest, capped amount to the Trust Score, tapering as verified job history accumulates. (SRS FR-35) TRU-11 The system shall award milestone badges (Founding Professional, tier upgrades, jobs milestones) with a celebratory unlock animation; Trust Score changes shall be event-driven, auditable, and shown as a trust history timeline to the owner. 3.5 Jobs & Applications, verification-gated (JOB) ID Requirement JOB-01 Posting a job shall be restricted to accounts at Tier 2+ (ID Approved); accounts below Tier 2 shall be shown exactly which verification steps unlock posting. This applies to individual and business posters alike, to reduce fake-job lures. (SRS FR-20, FR-21) JOB-02 Job posting shall use a multi-step stepper (Details → Location → Budget → Review) with progress, autosave/drafts, and photo attachments, capturing title, category (from the admin taxonomy), description, location (map + current-location), budget range, urgency, and an encouraged fixed-price quote/range. (SRS FR-23) JOB-03 The system shall show a job’s applicants/applications with worker previews and allow inviting specific workers. JOB-

04 Workers shall see incoming Job Requests and matched jobs and apply (with an application/quote), tracking status (submitted, under review, shortlisted, not selected, hired). JOB-05 The system shall model a job lifecycle: Draft → Open → Quoted → Accepted → In Progress → Completed → Released, plus Cancelled, Disputed, Archived (state table in Part III, §21.3). JOB-06 A live Job-in-Progress view shall show a status timeline (accepted, on the way, in progress, completed, payment released) with contact actions. JOB-07 On completion the system shall confirm, trigger escrow release on client confirmation, and prompt a review; job editing, drafts, cancellation rules, and an archive shall be supported, each list with empty/loading states. 3.6 Quotes & Negotiation (QTE) ID Requirement QTE-01 Workers shall send Quotes/Proposals for a job specifying fixed price, estimated duration, an itemised “what’s included”, a message, and attachments. QTE-02 Clients shall view received quotes with worker context (Trust Score, rating, availability) and Accept, Decline, Chat, or request a counter-offer. QTE-03 The system shall track quote states (Draft, Sent, Viewed, Accepted, Declined, Expired) and support counter-offers; accepting creates a booking and initiates escrow at the agreed price. QTE-04 Workers and clients shall each have quote inboxes (Sent / Received / Drafts) with status chips and timestamps. 3.7 Messaging (MSG) ID Requirement MSG-01 The system shall provide real-time 1:1 conversations tied to jobs/quotes, with an inbox (All/Clients/System/Offers), search, unread counts, and typing/online indicators. MSG-02 Messages shall support text, images, voice notes (record/play with waveform), location sharing, quote cards, job cards, and payment requests, and shall surface escrow/job status inline. MSG-03 The system shall support delivery/read status, block user, and report conversation; the inbox and conversation shall have empty states and be readable offline for recently synced threads (see SYS). MSG-04 The messaging schema (Conversation / Message / Attachment with nullable participants) shall support future agent assignment and admin oversight without a rewrite. 3.8 Payments, Escrow, Wallet & Payouts (PAY) ID Requirement PAY-01 The system shall support secure in-app payment via Campay, holding funds in escrow from acceptance/payment until the Client confirms completion, then releasing to the Worker minus the platform fee. (SRS FR-36) PAY-02 The system shall present an Escrow timeline: Paid → Accepted → In Progress → Completed → Released, with a “funds are secure” assurance; a ~10% service fee is shown transparently before payment. PAY-03 Disputed jobs shall freeze the TrustProciated escrowed funds pending Admin resolution; all payment and escrow events shall be written to an auditable, tamper-evident trail. (SRS FR-37, FR-38) PAY-04 Campay shall be integrated for collection and payout via Mobile Money (MTN, Orange), card, and bank in XAF; users shall manage Payment Methods with a default and secure storage. PAY-05 The Wallet shall show available balance, escrow overview (in-escrow amount), Add Money, Withdraw, and a last-transaction summary; a Transaction History (payments, holds/releases, refunds, withdrawals, fees) shall provide filters, status chips, and per-item detail. PAY-06 Payment and escrow transitions shall be verified server-side against Campay webhooks/callbacks and be idempotent; failures shall show explicit plain-language reasons (declined, insufficient funds, operator timeout with retry), never a silent failure. 3.9 Reviews & Ratings (REV) ID Requirement REV-01 After a completed job the client shall rate the worker (1–5 stars), write a review, and optionally attach photos; the worker may review the client. REV-02 Profiles shall show a reviews summary (average, distribution bars, count) and a full list; reviews shall be tied to real completed jobs to prevent fabrication and shall feed the Marketplace Trust sub-score. REV-03 The system shall allow reporting of reviews; admins may moderate per policy. 3.10 Safety & Emergency (SOS) Safety is a trust baseline Although SOS ranked 4th in the survey (29.6%), safety is treated as a trust-building baseline rather than deprioritised by vote share; users often under-report its importance until a need arises. (SRS FR-39) ID Requirement SOS-01 The system shall provide an Emergency/SOS control (press-and-hold), accessible during an active job engagement, that shares live location with the user’s emergency contacts and/or the platform safety process. (SRS FR-39) SOS-02 Users shall manage Emergency Contacts and view a Safety Center; the system shall record an SOS History (timestamped alerts with location and resolution: resolved / false alarm). SOS-03 The system shall support optional live tracking of an in-progress job (worker en route) with ETA/distance, gated by explicit consent; SOS shall be reachable from primary navigation in minimal taps on mobile. 3.11 Notifications (NOT) ID Requirement NOT-01 The system shall send web push and/or email (SMS later, behind the channel service) for key events: new job matches, quote requests, messages, verification status changes, and payment/escrow events. (SRS FR-46) NOT-02 A Notification Center shall provide All/Unread/Important filters, mark-as-read, per-channel preferences (in-app / web push / email; SMS shown as coming soon), detail views, deep-links, and an empty state. 3.12 Business & Team (BIZ) ID Requirement BIZ-01 Job posts marked as coming from a registered business shall support optional enhanced Business Verification (RCCM/registration) granting a “Verified Business” badge and eligibility for Tier-3-gated corporate engagements. (SRS FR-22) BIZ-02 A Business Dashboard shall summarise active posts, applications, in-progress/completed jobs, spending, and recent payments; businesses shall post team/bulk jobs (team size, duration, location). BIZ-03 Businesses shall manage Team Members with roles (admin, HR manager, accountant, staff), access levels (full / manage jobs / payments / view-only), status, and staff invitations, plus a company wallet and business analytics (jobs posted, hire rate, spend, average rating). 3.13 Agent Operations (AGT) (Invited by admin via email) ID Requirement AGT-01 Field Agents shall have a lightweight, mobile-friendly intake tool to register and immediately badge a Founding Professional recruited in person. (SRS FR-44, supports FR-31) AGT-02 An Agent Dashboard shall show tasks today, verifications done, approvals, pending reviews, field visits, today’s schedule, and a coverage-area map. AGT-03 A Verification Pipeline (Submitted → Under Review → Field Check → Pending Approval → Approved) shall show counts, per-item review, average review time, and approval rate; agents shall approve/reject with reasons or “request more info”. AGT-04 Agents shall onboard workers in person, capture/upload documents, run field verification, review portfolios, inspect businesses, manage assigned areas/routes, and start a mapped route; agent performance and daily activity shall be tracked. 3.14 Admin Operations (ADM) ID Requirement ADM-01 An Admin Dashboard shall show platform KPIs (total users, verified workers, jobs completed, total revenue with deltas), platform-performance charts, jobs-by-category, escrow overview, disputes overview, SOS alerts, and system health. ADM-02 Admins shall create, edit, and retire the taxonomy of service categories that search and job posting are scoped to. (SRS FR-40) ADM-03 Admins shall review and approve/reject submitted ID and business-registration documents, capturing reasons for rejection (feeding the friendly-retry UX in TRU-07). (SRS FR-41) ADM-04 Admins shall view and, in exceptional cases, adjust Trust Scores (e.g. apply a fraud penalty or a Field-Verified bonus for a Founding Professional). (SRS FR-42) ADM-05 Admins shall have a dispute-resolution workspace covering escrowed payments, cancellations, and reported incidents, with details, messages, evidence, history, and actions (release payment, refund client, request info, close). (SRS FR-43) ADM-06 Admins shall manage Users/Workers/Businesses/Agents; oversee Payments & Escrow and Reviews moderation; and operate a Fraud & Risk monitor (high-risk users, suspicious jobs, blocked accounts, risk alerts) plus a platform map/heatmap. Invite agents via email ADM-07 Admins shall access Analytics & Reports (user growth, jobs over time, jobs by category/location, revenue by source, trust-score distribution, top performers) with export, plus Audit Logs, Announcements, Roles & Permissions (RBAC), Integrations, and System Settings. 3.15 Settings & Account Management (SET) ID Requirement SET-01 All users shall view/edit their profile, manage uploaded documents/photos, and view their current Tier and Trust Score with an explanation of how to improve it. (SRS FR-45) SET-02 Users shall manage password/security (2FA), payment methods, notification preferences, language, theme, connected devices, and account deletion. SET-03 The system shall provide an Invite & Earn referral flow (unique code/link, earnings, friends joined) and Help & Support from primary navigation. 3.16 Platform, PWA, i18n & System States (SYS) ID Requirement SYS-01 The Application Shell shall be independently installable as a PWA (own manifest and service worker) distinct from the Marketing Landing Page, which is intentionally excluded from service-worker scope to stay a fast, crawlable SEO page. (SRS FR-3, §3.5) SYS-02 The PWA shall cache the app shell and last-viewed data (profiles, jobs) with stale-while-revalidate for offline/degraded viewing, and shall queue write actions to sync on reconnect, with a clear offline indicator. (SRS NFR-10) SYS-03 The UI shall be fully bilingual (English/French) via i18n, including dates, numbers, and XAF currency formatting; no hardcoded user-facing strings. SYS-04 The system shall provide consistent system states: 404, 500, offline/no-connection, maintenance, loading (skeletons, not spinners), empty states, and success states. SYS-05 The marketing site shall be SEO- and conversion-optimised (metadata, canonical, OG/Twitter, structured data, sitemap, robots); public pages shall be statically rendered or ISR. SYS-06 Every mutating form shall show inline success/error banners and field-level validation; no silent failures. 4. Non-Functional Requirements NFRs are tagged NFR-- and are testable. They encode the platform’s constraints for a mobile-first, bilingual, low-bandwidth, trust-critical audience. 4.1 Performance ID Requirement NFR-PERF-01 Marketing pages shall meet Core Web Vitals: LCP < 2.5s, CLS < 0.1, INP < 100ms on a mid-range mobile over 3G/4G. NFR-PERF-02 Discovery and dashboard reads shall render meaningful content in < 1.5s on warm cache; long lists shall be virtualised and paginated. NFR-PERF-03 GraphQL reads shall let clients request exactly the fields needed to minimise payload over mobile data; images shall be responsive and lazily loaded. NFR-PERF-04 No N+1 queries; frequent query combinations shall be backed by indexes; slow queries (>300ms) shall be logged at warn level. NFR-PERF-05 Animations shall use transform/opacity only (never layout properties) and target 60fps; animation logic is centralised in a motion utility module and runs at 60fps on phone browsers. 4.2 Scalability & Availability ID Requirement NFR-SCAL-01 Backend

services shall be horizontally scalable, stateless, and containerised; sessions are JWT-based (no server affinity). NFR-AVAIL-01 Target availability 99.5%+ for the application API; graceful degradation when non-critical dependencies (maps, analytics) are unavailable. NFR-SCAL-02 Spatial tile generation (CARTO/MVT) and heavy analytics shall not block transactional paths. NFR-AVAIL-02 Payment and escrow operations shall be idempotent and reconcilable against Campay callbacks after transient failures. 4.3 Security & Privacy ID Requirement NFR-SEC-01 All traffic over TLS; secrets in a managed store; passwords hashed (bcrypt/argon2); JWTs short-lived with refresh. NFR-SEC-02 RBAC shall enforce role/permission boundaries server-side for every write and privileged read; clients never trusted for authorisation. NFR-SEC-03 All inputs validated (Zod/DTOs) before touching the database; public APIs rate-limited; forms carry honeypots; abuse throttled. NFR-SEC-04 Identity documents, selfies, and KYC media shall be access-controlled, encrypted at rest, and visible only to the owner and authorised agents/admins. NFR-PRIV-01 PII and location data collection shall be consented and minimised; live location/tracking requires explicit opt-in and is revocable. NFR-SEC-05 Financial and verification actions shall be audit-logged with actor, timestamp, and request ID. 4.4 Usability, Accessibility & Localization ID Requirement NFR-UX-01 The app shall be mobile-first and touch-optimised; primary actions reachable within thumb range; SOS reachable in minimal taps. NFR-A11Y-01 The UI shall meet WCAG 2.1 AA: keyboard operability, focus management/traps in overlays, ARIA roles, sufficient contrast, and prefers-reduced-motion support. NFR-I18N-01 Full English/French parity including formatting of currency (XAF), dates, and numbers; no hardcoded user-facing strings. NFR-UX-02 Loading uses skeletons (not spinners); every list/search has a branded empty state; every mutating form gives inline feedback. 4.5 Reliability, Offline & PWA ID Requirement NFR-PWA-01 The app shell shall be installable and cached; core read surfaces shall render last-synced content offline. NFR-PWA-02 Actions taken offline (e.g. drafting) shall queue and sync on reconnect with conflict-safe, last-write-wins semantics for simple fields. NFR-REL-01 Mobile Apollo cache shall persist to secure device storage for instant offline-first rendering when connectivity drops. 4.6 Observability & Maintainability ID Requirement NFR-OBS-01 A centralised, structured logger shall emit JSON logs in production with scope, request ID, duration, and sanitised payload/result summaries (never secrets). NFR-OBS-02 Each incoming request shall carry a unique request ID propagated through logs and returned in error responses for support. NFR-MNT-01 All database access goes through Prisma (raw SQL only for PostGIS spatial ops via reviewed views/queries); migrations run at deploy time only, never per request. NFR-MNT-02 Shared types, GraphQL operations, and the Prisma client shall live in workspace packages consumed by web, mobile, and API. 5. External Interface Requirements 5.1 User interfaces • Two experiences: emotional/animated marketing site and productive/calm application, both built from the TMDS token system (Part III). • Web navigation: sidebar + top search + notifications + role switch + profile (desktop); bottom navigation Home/Jobs/Messages/Wallet/Profile (mobile). • Admin and agent surfaces are data-dense, productivity-first (admin uses a dark, chart-heavy interface). • All colour, spacing, radius, type, motion, and component behaviour derive from tokens; light and dark themes both shipped. 5.2 Software interfaces (external services) Service Role in TrustPro Campay Payments: Mobile Money (MTN/Orange), card, bank, collection, escrow funding, and payout in XAF; webhooks confirm state. OpenStreetMap (OSM) Authoritative baseline geodata: administrative boundaries, roads, POIs, extracted into PostGIS. PostGIS Spatial storage/indexing (GIST) and queries over OSM geometry and live coordinates. CARTO Spatial analytics processor over PostGIS; generates dynamic Mapbox Vector Tiles (geofencing, hot-spots, driver distribution). Mapbox GL JS / Native Presentation only, hardware-accelerated rendering and custom styling of tiles; never authoritative for spatial math. BigDataCloud Fast server-side reverse-geocoding (administrative truth). AWS S3 Object storage for portfolio media, documents/KYC, and voice notes. Auth.js Stateless JWT session management across web and installed PWA instances. Notification channels Web push (VAPID) + branded email at launch; SMS provider slot stubbed behind the NotificationChannel service (flag OFF). Phone numbers stored/normalised (E.164) so SMS switches on later with zero call-site changes. 5.3 Communication interfaces • GraphQL over HTTPS for all read/query traffic (Apollo client ↔ NestJS resolvers). • REST over HTTPS for all writes/mutations, file uploads, and webhooks (NestJS controllers). • WebSocket (NestJS gateways) for real-time messaging, live job status, notifications, and location telemetry. • Signed callbacks/webhooks from Campay for payment and payout confirmation. 6. Data Requirements (Domain Model) The core entities below drive the Prisma schema (Part II, §11). Relationships are summarised; indexes and constraints are specified in the architecture section. Spatial columns (geometry/geography) are managed in PostGIS and accessed via reviewed raw SQL/views rather than Prisma field access. 6.1 Core entities Entity Key attributes Notable relations User id, phone, email, passwordHash, name, avatar, locale, status, createdAt has many UserRole; has Wallet; has EmergencyContacts UserRole userId, role (WORKER|CLIENT|BUSINESS|AGENT|ADMIN), isActive belongs to User WorkerProfile userId, headline, bio, tierId, trustScore, onTimePct, location (geo) has Services, PortfolioItems, Availability, Documents, Reviews ClientProfile userId, defaultLocation (geo) belongs to User BusinessProfile ownerId, name, type, rccm, description, verifiedAt, location (geo) has TeamMembers, Jobs, Documents AgentProfile userId, coverageRegionId, qualityScore has Assignments; performs Verifications ServiceCategory id, name(en/fr), icon, workerCount has Services Service id, workerId, categoryId, name, description, startingPrice, isActive belongs to WorkerProfile & ServiceCategory PortfolioItem id, workerId, type (photo|video|project), mediaKey, caption, group belongs to WorkerProfile Availability id, workerId, dayOfWeek, startTime, endTime, exceptions[] belongs to WorkerProfile TrustScore id, subjectId, subjectType, identity, professional, marketplace, financial, total 1:1 with Worker/Business; has TrustEvents VerificationRecord id, subjectId, checkType, status, reviewedBy, reviewedAt, reason belongs to subject; linked to Agent Document id, ownerId, kind (id|selfie|address|cert|portfolio|business), s3Key, status, visibility belongs to owner Endorsement id, subjectId, authorId, relationship, quote, rating belongs to subject & author Job id, clientId|businessId, categoryId, title, description, location (geo), budgetMin, budgetMax, urgency, status has Applications, Quotes, Booking, Photos JobApplication id, jobId, workerId, status, message belongs to Job & Worker Quote id, jobId, workerId, price, durationEst, includes[], message, status belongs to Job & Worker Booking id, jobId, quoteId, workerId, clientId, agreedPrice, status 1:1 with accepted Quote; has EscrowHold Conversation id, jobId?, participantAId, participantBId, lastMessageAt has Messages Message id, conversationId, senderId?, kind (text|image|voice|location|quote|job|payment), body/mediaKey, status belongs to Conversation Wallet id, ownerId, balance, currency=XAF has Transactions; has PaymentMethods PaymentMethod id, walletId, kind (mtn|orange|card|bank), maskedRef, isDefault belongs to Wallet Transaction id, walletId, type (payment|escrow_hold|escrow_release|refund|withdrawal|fee), amount, status, ref belongs to Wallet; links Booking EscrowHold id, bookingId, amount, feeAmount, state (paid|accepted|in_progress|completed|released|refunded) 1:1 with Booking Review id, jobId, authorId, subjectId, rating, text, photos[] belongs to Job; feeds TrustScore Dispute id, jobId, openedById, reason, status (open|escalated|resolved), resolution belongs to Job; has messages/evidence Notification id, userId, type, payload, channel, readAt belongs to User SOSAlert id, userId, location (geo), status (active|resolved|false_alarm), createdAt belongs to User; notifies contacts EmergencyContact id, userId, name, phone belongs to User LocationPing id, subjectId, jobId?, geo, accuracy, source, ts telemetry for tracking (PostGIS) GeoRegion id, name, boundary (geo), parentId administrative area from OSM TeamMember id, businessId, userId, role, accessLevel, status, joinedAt belongs to BusinessProfile Referral id, referrerId, code, refereeId?, rewardAmount, status belongs to User AuditLog id, actorId, action, target, requestId, meta, ts cross-cutting 6.2 Data integrity rules • A Review must reference a Completed job and its two participants (prevents fabricated reviews), feeds Marketplace Trust. • An EscrowHold transitions only via the defined state machine (Part II); “released” requires client confirmation or admin resolution. • Trust Score sub-scores are derived/append-only via TrustEvents; the total is recomputed, never edited directly. • Verification status changes are attributed to an Agent/Admin and audit-logged. • Location writes carry a source (device GPS, human input, reverse-geocode) so human input can override machine fallback (Part II, §12). PART II System Architecture 7. Architecture Overview TrustPro is a TypeScript-first monorepo with three deployable apps (web, mobile, API) and shared packages, over a PostgreSQL/PostGIS core. The API exposes a deliberate dual surface: GraphQL for all reads and REST for all writes. Spatial computation is delegated to PostGIS and CARTO; maps are pure presentation. The design is chosen for mobile-data efficiency, offline-first resilience, and geospatial performance. 7.1 High-level component view (Mermaid) flowchart TB subgraph Clients W[Next.js Web / PWA] M[Installed PWA instance] end subgraph Edge V[Vercel Edge / CDN] end subgraph API[NestJS API - Bun] GQL[GraphQL Resolvers - reads] REST[REST Controllers - writes] WS[WebSocket Gateways] end subgraph Data PG[(PostgreSQL + PostGIS)] S3[(AWS S3)] end subgraph Spatial OSM[OpenStreetMap] –> PG CARTO[CARTO -> MVT tiles] MB[Mapbox render] BDC[BigDataCloud reverse-geocode] end CAM[Campay payments] W –>|reads GQL| GQL W –>|writes REST| REST M –>|reads GQL| GQL M –>|writes REST| REST W & M <–>|realtime| WS WS –> V GQL & REST –> PG REST –> S3 PG –> CARTO –> MB REST –> BDC REST <–>|webhooks| CAM WS –> PG 8. Package & Workspace Architecture A root pnpm-workspace.yaml orchestrates apps and shared packages with deterministic symlinking. Both apps are executed on the Bun runtime (bun dev, bun run build) for fast compilation and server rendering; Turborepo runs workspace tasks (dev/build/lint/type-check) with caching. Husky runs pre-commit checks. 8.1 Repository blueprint TrustPro/ ├─ apps/ │ ├─ web/ # Next.js (App Router, Bun), marketing + app + Mapbox GL JS │ └─ api/ # NestJS (Bun), REST controllers + GraphQL

resolvers + PostGIS ├─ packages/ │ ├─ api-client/ # shared client config, GraphQL queries, mutation hooks │ ├─ database/ # Prisma client, migrations, PostGIS/OSM schema views │ └─ typescript-config/ # shared tsconfig blueprints ├─ package.json ├─ pnpm-workspace.yaml └─ pnpm-lock.yaml Concern Decision Orchestration pnpm workspaces; strict, deterministic node_modules symlinking. Task runner Turborepo (turbo run dev/build/lint/type-check) with local caching. Web/API runtime Bun (bun dev, bun run build) for compilation and SSR speed. Shared code Prisma client, GraphQL operations, and TS types live in packages/*. Type safety Prisma emits a shared client consumed across all apps. 9. Frontend Architecture 9.1 Web (apps/web) • Next.js App Router (TypeScript), executed via Bun, SSR/SSG/ISR + API routes unified; marketing pages static/ISR, app pages dynamic. • TailwindCSS + shadcn/ui, shadcn source is copied into the workspace and fully re-styled to TMDS (never default appearance). • Anime.js v4, tree-shakeable timeline orchestration for hero motion, scroll storytelling, and micro-interactions; Lenis for smooth scroll. • Mapbox GL JS, renders CARTO-served vector tiles with custom TMDS styling. • Forms/i18n/theme, React Hook Form + Zod, next-intl (en/fr), next-themes (light/dark), TanStack Virtual for long lists. 9.2 Installable PWA shell (web-only) • The app ships its own manifest + service worker (separate from the marketing landing, which is excluded from SW scope) and installs on phone and desktop. • App-shell caching + stale-while-revalidate keep previously viewed profiles, jobs, and synced messages browsable offline behind a clear offline indicator; offline writes queue and sync on reconnect. • Web push via VAPID (permission UX explains value before the browser prompt); push events and notification clicks are handled in the service worker and deep-link into the app. • Location uses the browser Geolocation API; real-time position is foreground streaming over WebSocket while the relevant screen is open, with explicit revocable consent (web has no reliable background tracking). 9.3 Client data layer, Apollo Client Apollo Client is the localized single source of truth for read-side data and the normalized cache. Writes bypass the GraphQL link entirely and go to REST via native fetch/axios. • Web, @apollo/experimental-nextjs-app-support streams GraphQL queries inside React Server Components before they reach the browser. • Offline persistence, apollo3-cache-persist commits the cache to IndexedDB (localforage) for instant offline-first rendering when connectivity drops. • Write bypass, all mutations (create/update/delete, uploads) use HTTP calls straight to REST endpoints; the read cache is updated optimistically and reconciled. 9.4 Rendering & motion rules • Public marketing pages are static (or ISR) by default; dynamic rendering only for authenticated/app/admin routes reading session. • app/global-error.tsx exists from init; every mutating form shows inline success/error feedback. • Motion profile by surface: marketing = strong staggered/directional reveals; content = light reveals; app/dashboard = minimal. All motion respects prefers-reduced-motion and uses transform/opacity only. • Sticky header transparent→solid on scroll; animated mobile drawer with focus trap and route-change close; scroll-margin-top offsets on all anchored sections. 10. Backend Architecture NestJS provides a strongly typed, modular backend with dependency injection and clean module boundaries, running natively on Bun. The same module can host both a REST controller (writes) and a GraphQL resolver (reads). 10.1 Dual API rule Surface Used for Rationale GraphQL (resolvers) All queries, indexing, data fetching (web + mobile). Clients request the exact payload shape, efficient over mobile data; feeds Apollo cache. REST (controllers) All mutations, state changes, file uploads, webhooks (POST/PUT/PATCH/DELETE). Simple business logic, validation piping, multipart media, and webhook handling. 10.2 Module map (NestJS) Module Responsibility auth JWT issue/refresh, OTP, 2FA, session guards, RBAC. users / roles Accounts, multi-role, role switching, RBAC policies. profiles Worker/Client/Business/Agent profiles, services, availability, portfolio. trust Trust Score computation, verification checks, tiers, endorsements, badges, trust events. discovery Search, filters, recommendations, nearby (delegates spatial to PostGIS/CARTO). jobs Job lifecycle, applications, bookings, state machine. quotes Quotes, counter-offers, acceptance → booking + escrow init. messaging Conversations, messages, attachments (REST writes + WS delivery). payments Wallet, payment methods, Campay collection/payout, escrow state machine, transactions, refunds. reviews Reviews tied to completed jobs; feeds trust. safety SOS alerts, emergency contacts, live tracking consent. notifications Event-driven dispatcher, NotificationChannel providers (web push/email, SMS stub), NotificationLog, localized template store, preferences. business Team members, access levels, invitations, company verification, business analytics. agents Assignments, verification pipeline, field ops, performance. admin Dashboards, verification queue, disputes, fraud & risk, categories, reports, audit, announcements, settings. geo OSM/PostGIS/CARTO integration, reverse-geocode, region resolution. realtime WebSocket gateways for messaging, job status, notifications, telemetry. 11. Data & Persistence • PostgreSQL is the system of record; PostGIS adds spatial storage, GIST indexing, and queries over OSM geometry and live coordinates. • Prisma is the ORM and emits a shared client (packages/database). Complex PostGIS operations bypass Prisma field access and run via reviewed raw SQL or database views. • Migrations run only at deploy time (prisma migrate deploy in CI before the server boots), never inside request handlers; migration_lock.toml committed with provider="postgresql"; .sql files plain UTF-8 (no BOM). 11.1 Indexing policy Targeted indexes back every frequent query combination; slow queries are logged. Representative examples: @@index([status, createdAt]) // job lists by state @@index([categoryId, status]) // discovery by category @@index([workerId, status]) // a worker's jobs/quotes @@index([walletId, createdAt]) // transaction history // PostGIS (raw): GIST index on geometry/geography columns for KNN & within queries 12. Spatial & Geographic Architecture Location is treated as probabilistic, layered, and human-augmented. Each layer has one job; none is asked to do another's. Layer Role OpenStreetMap Structural reality, roads, buildings, boundaries, base features extracted into PostGIS. PostGIS Warehouse, stores/indexes OSM geometry and live operational coordinates; authoritative spatial math. CARTO Kitchen scientist, runs complex spatial SQL on the fly (geofencing, hot-spots, distributions) and emits dynamic MVT tiles. BigDataCloud Fast server-side reverse-geocoding for administrative truth. Mapbox GL / Native Presentation plate, hardware-accelerated styling/rendering of CARTO tiles; never authoritative for calculations. Human input Landmarks, plus-codes, explicit notes, in-app chat, permanently overrides machine fallback positions. Real-time tracking Browser Geolocation API (foreground, consented) → NestJS WebSocket gateways → PostGIS. Rule Mapbox is never authoritative for spatial calculations. Distance, proximity, geofencing, and clustering are computed in PostGIS/CARTO; Mapbox only paints the result. flowchart LR OSM[OSM extract] –> PG[(PostGIS)] Dev[Device GPS] –> WS[WS gateway] –> PG Human[Human input] –> PG PG –> CARTO[CARTO spatial SQL] –> MVT[MVT tiles] –> MB[Mapbox render] REST –> BDC[BigDataCloud reverse-geocode] –> PG 13. Real-Time Architecture • NestJS WebSocket gateways power messaging delivery, live job status, notification push, and location telemetry ingestion. • The browser streams foreground geolocation over the WS gateway while the tracking screen is open; the gateway persists pings to PostGIS and fans out live-tracking updates to consented viewers. Redis backs the WS adapter so gateways scale horizontally. • The messaging schema (Conversation/Message/Attachment) supports future admin oversight and agent assignment without rework. 14. Authentication & Security Architecture • Auth.js manages stateless JWT sessions; refresh tokens rotate; secrets in a managed store. Auth code stays OFF Edge routes (Node runtime only): Auth.js/bcrypt/jsonwebtoken use Node APIs incompatible with the Edge runtime. • RBAC guards enforce role/permission on every write and privileged read; the Role Switcher changes the active role claim, not identity. • All inputs validated with Zod/DTOs before the database; public routes rate-limited; honeypots on public forms; audit logging on financial and verification actions. • KYC media is access-controlled and encrypted at rest; location/tracking is explicit opt-in and revocable. 15. Payments & Escrow Architecture Campay is the payment aggregator for Mobile Money (MTN, Orange), card, and bank in XAF. Escrow is a server-side state machine; the client UI mirrors it as the escrow timeline. All transitions are idempotent and reconciled against Campay callbacks. 15.1 Escrow state machine stateDiagram-v2 [*] –> Paid: client funds via Campay (held) Paid –> Accepted: worker accepts booking Accepted –> InProgress: work starts InProgress –> Completed: worker marks done Completed –> Released: client confirms -> payout to worker (minus fee) Paid –> Refunded: cancel before acceptance InProgress –> Disputed: either party disputes Disputed –> Released: admin resolves for worker Disputed –> Refunded: admin resolves for client Released –> [*] Refunded –> [*] • A ~10%(admin configurable) platform service fee is computed transparently and shown before payment; payout = agreed price − fee. • Campay webhooks confirm collection and payout; handlers are idempotent (keyed by external ref) and audit-logged. • Payment failures surface explicit reasons (declined, insufficient funds, operator timeout with retry), never silent. 16. Storage & Media • AWS S3 stores portfolio photos/videos, KYC documents/selfies, voice notes, and business documents. • Uploads go through REST (multipart) with validation and virus/type checks; access is signed and scoped by ownership/role. • Media is served responsively; images are optimised and lazily loaded for low-bandwidth clients. 17. Notifications Architecture (dispatcher pattern) • Controllers emit, never send: business modules emit typed domain events (@nestjs/event-emitter); a NotificationDispatcher subscribes, applies per-user preferences, selects channels by configurable priority (Push → Email → SMS when enabled), and delegates to providers behind the NotificationChannel interface (WebPush via VAPID, branded Email, stubbed SMS behind a feature flag). Adding a channel never touches business logic. • Every dispatch attempt (success or failure) is written to a NotificationLog (channel, provider, recipient, templateKey, status, errorMessage) with indexes for failed-delivery dashboards; delivery is queued and retried via BullMQ on Redis. • Templates live in a central localized store keyed by templateKey with EN/FR variants per channel (email HTML, push title/body, SMS text); never hardcoded at dispatch sites. Emails are

brand-designed (tokens, logo, footer), never plain. • Per-user preferences (channel × category) are enforced by the dispatcher; security-critical notices (password reset, new-device login) bypass preferences and always use email. Web push uses a multi-device registry (endpoint, deviceName, lastSeen, isActive) with expired-subscription cleanup; VAPID keys are generated once and never rotated casually. • Phone numbers are normalised to E.164 (+237…) at capture so SMS can switch on later cleanly. • Events include job match, quote sent/accepted, payment received/released, review received, verification updates, disputes, and SOS. • Delivery is queued and retried; each notification deep-links to the relevant surface. 18. Deployment & Infrastructure Component Platform Web (Next.js) Vercel, edge-native optimisation for App Router routing. Redis Upstash / Railway Redis, BullMQ queues (notifications, webhooks), rate-limit store, WS pub/sub adapter. Backend (NestJS/Bun) Dockerised containers on Railway / Fly.io / Render. Database + spatial Supabase / Neon (serverless Postgres with PostGIS) integrated with a CARTO cloud workspace. Object storage AWS S3. Payments Campay. 18.1 Deployment gates (from engineering standards) • prisma migrate deploy succeeds before the server starts; migration_lock.toml committed; .sql files UTF-8 (no BOM). • global-error.tsx present; all public pages static or ISR; SMTP/notification timeouts configured. • Rate limiting on all public APIs; auth errors accurate (credentials ≠ session); Campay webhook idempotency verified. 19. Observability, Logging & Migrations • A centralised logger (lib/logger) emits JSON in production with level, scope, message, requestId, durationMs, and sanitised payload/result summaries; raw Prisma query logs are off unless explicitly opted in during development. • A unique requestId is generated per request in the first middleware, propagated through all logs, and returned in error responses. • Log-level policy: debug (dev only), info (request in/out, DB done), warn (slow query >300ms, rate-limit approaching), error (DB/payment/unhandled). • N+1 avoided via include/select and pagination on all list endpoints; migrations never run at request time. PART III Design System, Trust Model & Delivery 20. Trust Motion Design System (TMDS) TMDS is a product language, not just a component kit. Every component must communicate trust, progress, transparency, or professionalism, organised around the four pillars so screens feel like one product. Style is “Modern Soft Premium”: soft surfaces, generous spacing, rounded corners, depth via shadow, beautiful typography, smooth purposeful motion, minimal icons; no neumorphism, no glass-everywhere, no gradient-everywhere, no skeuomorphism. 20.1 Colour tokens Token Hex Use Primary, Deep Indigo #3D5AFE Trust, technology, reliability; primary brand. Success #14B86A Trust Score, verified, completed, escrow. Accent #00C2A8 Interactions, buttons, progress, hover. Warning #F59E0B Cautions, pending states. Danger #EF4444 Errors, destructive, disputes. Neutrals Gray 50–900 Surfaces and text (close to Tailwind neutral). Both light and dark themes ship; neutrals avoid pure white/black extremes. All values are tokens, with no hardcoded colours anywhere. 20.2 Typography Role Family Notes Headings Geist or Inter Tight Weights 900/700/600; sizes 64/48/36/24. Body Inter Weights 500/400; 18/16. Numbers IBM Plex Sans Trust Score, money, statistics. 20.3 Spacing, radius, shadow • Spacing (8pt grid): 4, 8, 12, 16, 24, 32, 40, 48, 64, 80, 96, 128; nothing arbitrary. • Radius: cards 20, inputs 16, buttons 14, images 24, profile photos circular. • Shadows: soft, not black, e.g. 0 10 40 rgba(0,0,0,.08), not heavy 0 20 60 rgba(0,0,0,.35). 20.4 Motion principles Animation should explain, not decorate; every motion answers “what changed?”. Skeletons over spinners; confetti only at onboarding completion. Interaction Motion Page transition ~450ms fade + scale + blur. Card hover Lift + shadow + scale 1.02. Button Ripple / gradient shift / micro-bounce. Loading Skeleton (never spinner). Success Checkmark draw; confetti only on onboarding completion. Trust Score Animated circular gauge, count-up, tier upgrade. Escrow Animated timeline stepper. 20.5 Component library (target: 120–180 components) Group Examples Buttons Primary, Secondary, Ghost, Icon, Loading. Inputs Text, Search, Phone, OTP, Password, Location. Cards Worker, Job, Portfolio, Review, Statistic, Verification. Navigation Top bar, Sidebar, Bottom bar, Breadcrumbs, Role Switcher. Trust Trust Score Ring, Verification Timeline, Tier Badge, Progress Tracker. Commerce Escrow Timeline, Quote Card, Payment Status, Wallet Summary. Discovery Filter Drawer, Map Preview, Category Chips, Search Suggestions. Feedback Skeletons, Toasts, Empty States, Success States, Error States. Overlays Modals, Drawers, Command Palette, Image Viewer. Data-viz (admin) Charts, Heatmaps, Analytics Cards. 20.6 Signature surfaces • Landing (conversion): massive hero with worker/client portraits, floating trust badge and animated Trust Score; social-proof count-ups (15,000+ verified, 98% success, 4.9★, ₣120M+ paid); Trust Engine section visualising score progression 15→35→60→82→100 on scroll; horizontal portfolio with before/after; escrow-explained pricing; testimonial with voice waveform; single-button dark CTA. • Discovery (the masterpiece): location selector + search + trust filter; featured/nearby/recommended/trending rails; map preview; expandable worker cards. • Worker Profile: cover, trust ring, portfolio, timeline, reviews, pricing, book/message, verification. • Verification: gamified progress with benefit unlocks that animate. • Admin: dark, data-dense, chart-heavy, productivity-first. 21. Behavioural Diagrams & Trust Model Diagrams are given as Mermaid (rendered in-repo) and as tables, covering the primary behavioural flows plus the exact tier and Trust Score definitions from the Trust Engine analysis. 21.1 Progressive verification (sequence) sequenceDiagram participant Wk as Worker participant App participant API participant Ag as Agent/Admin Wk->>App: submit ID, selfie, address, portfolio App->>API: REST upload (S3) + create VerificationRecords(pending) API–>>Ag: appears in Verification Pipeline Ag->>API: field check + approve/reject(+reason) API->>API: emit TrustEvents -> recompute Trust Score & Tier API–>>Wk: notification: badge unlocked / tier upgraded 21.2 Job to escrow to completion (sequence) sequenceDiagram participant Cl as Client (Tier 2+) participant Wk as Worker participant API participant Cam as Campay Cl->>API: post job (REST) -> Open Wk->>API: send quote (REST) Cl->>API: accept quote -> Booking + EscrowHold(Paid) API->>Cam: collect funds (MoMo/card) Cam–>>API: webhook confirmed (idempotent) Wk->>API: accept -> InProgress -> mark Completed Cl->>API: confirm completion API->>Cam: payout worker (price - fee) -> Released API–>>Cl: prompt review; API–>>Wk: trust update 21.3 Job lifecycle (state table) State Entered when Exits to Draft Client saves an unfinished post Open (publish), Archived Open Job published Quoted, Cancelled Quoted ≥ 1 quote received Accepted, Cancelled Accepted Client accepts a quote (escrow Paid) In Progress, Refunded In Progress Worker starts work Completed, Disputed Completed Worker marks done Released, Disputed Released Client confirms (payout) terminal Disputed Either party disputes Released, Refunded (admin) Cancelled / Refunded / Archived Cancellation / resolution / close-out terminal 21.4 Progressive verification tiers (exact gates) Tier Requirements to reach Capabilities unlocked Tier 0, Basic Phone (OTP), full name, profession, profile photo, current location, short bio Appears in search; max 2 active job requests/week; profile labelled “Basic Profile” (no deceptive implied trust) Tier 1, Phone + ID Submitted Tier 0 + national ID uploaded Unlimited active job requests Tier 2, ID Approved Tier 1 + ID approved by admin + portfolio uploaded “Verified Professional” badge; ranks above Tier 0 in search; eligible to post jobs as a Client Tier 3, Field Verified Tier 2 + in-person/field verification + min. 20 completed jobs + 4.8★+ rating Premium search placement; eligible for high-value and corporate/business contracts Tiers are derived from the Trust Score plus completed checks, never set manually; thresholds are Admin-configurable. Verification uses soft restrictions (e.g. missing ID limits job volume) rather than hard account blocks. 21.5 Trust Score reference (point model) In addition to discrete tiers, each user has a numeric Trust Score (0–100) for finer-grained, at-a-glance comparison, visibly displayed on profiles and factored into default search ranking. Event Score impact Starting score (new account) 15 Phone verified +10 ID verified +20 Field verified +15 Portfolio added +10 Completed jobs +15 Repeat customers +10 Excellent reviews +15 Disputes −20 Cancelled jobs −10 Fake documents Permanent suspension (overrides score) 21.6 The four trust pillars (scoring reference) Pillar Core question Signals Identity Trust Are you who you claim to be? Phone OTP, ID, selfie/liveness, GPS, address Professional Trust Can you actually do the work? Portfolio, field verification, certifications, verified past projects Marketplace Trust Do clients have good experiences with you? Reviews, repeat customers, completion rate, cancellation rate, response time, disputes Financial Trust Can money safely flow through the platform? Escrow via Campay, audit logs, fraud detection, chargeback/dispute history, payment reliability Endorsements Community endorsements (previous customer, workshop owner, quarter chief, landlord, community leader) contribute a modest, capped amount to the Trust Score that tapers in influence as verified platform job history accumulates, so early credibility is possible without diluting earned reputation. (SRS FR-35) 22. Feature Prioritisation (MoSCoW) Prioritisation blends the team meeting’s mandatory functionality with survey-validated demand. Priority Features Must Have Unified discovery (worker profiles + jobs + search + location setter + post-job CTA); guest browsing with login-gated actions; role switch (Worker ↔ Client); fuzzy/debounced search with admin-configured categories and “no provider yet” fallback; advanced filtering (location, price, tier, rating); tiered verification (Tiers 0–3) + Trust Score; verification-gated job posting (Tier 2+); worker portfolio photos (59.3%); visible Verified ID/Trust Score badge (59.3%); fixed-price quotes (51.9%); secure escrow via Campay (44.4%); PWA installability with separate marketing landing; baseline security hardening, responsive design, profile management. Should Have Emergency/SOS button (safety-critical despite 29.6% rank); community endorsements; ratings & reviews; web push + email notifications (SMS later); business verification badge for corporate posters. Could Have Live tracking of worker arrival (14.8%); corporate / Tier-3-gated high-value contracts; Field Agent “Founding Professional” recruitment tooling. Won’t Have (Phase 1) Native iOS/Android apps, explicitly deferred in the original SRS; the API is designed to add them without rearchitecting, and the delivery plan (§23) schedules them as the next phase. 23. Traceability & Delivery 23.1 Requirements traceability (summary) Each functional module maps to backend modules and primary surfaces (duplicate mockups merged into one canonical

screen per feature). FR module Backend module(s) Primary surfaces ACC auth, users, roles Landing CTA, guest browse, sign-up, OTP, role select, role switcher DIS discovery, geo Discovery home, location setter, filters, map/list, categories PRO profiles Worker profile, services, availability, portfolio, business profile TRU trust, agents Trust Score ring/breakdown, verification, documents, endorsements JOB jobs Post-job stepper (Tier 2+ gate), applications, job-in-progress QTE quotes Quote inboxes, send/accept, counter-offer MSG messaging, realtime Inbox, conversation, voice/quote/payment cards PAY payments Wallet, escrow timeline, payment methods, transactions REV reviews Review submit, profile reviews summary SOS safety, realtime SOS button, safety center, SOS history, live tracking NOT notifications Web push + email + notification center + preferences (SMS deferred) BIZ business Business dashboard, team, company verification, analytics AGT agents Agent dashboard, pipeline, routes, inspections, founding intake ADM admin Dashboard, categories, verification queue, disputes, fraud, reports SET / SYS users, admin, platform Settings, invite & earn, PWA/offline, system states 23.2 Suggested delivery phasing A build order that front-loads the shell, tokens, and trust primitives so nothing is retrofitted. The entire delivery is the Web PWA. 1. Foundation. Monorepo (pnpm + Bun), Prisma schema + PostGIS, shared packages, TMDS tokens, theme + animation systems, logger, global-error, CI with migrate-deploy gate and clean security scan. 2. Access & roles. Guest browse + login-gated actions with intent resume, phone/OTP sign-up, JWT sessions, RBAC, role switcher, onboarding shells, separate marketing landing. 3. Trust engine. Documents + verification records, tiers 0–3, Trust Score point model, badges, endorsements, agent pipeline, founding-professional intake. 4. Discovery & profiles. Location setter, fuzzy/debounced search over admin taxonomy, filters, worker card, profiles, portfolio, availability, spatial (PostGIS/CARTO/Mapbox). 5. Transactions. Job stepper (Tier 2+ gate) + lifecycle, quotes, bookings, Campay escrow state machine, wallet, transactions, disputes. 6. Messaging, safety & notifications. Conversations with voice/quote/payment cards, WS delivery, SOS + emergency contacts, live tracking, push/SMS notifications. 7. Reviews, business & admin. Reviews feeding trust, team/business, and full admin operations (categories, verification queue, disputes, fraud, reports). 8. PWA hardening + marketing. Conversion landing, SEO, offline/PWA queueing, i18n parity, empty/system states, accessibility, performance budgets. Using this document with Claude Code This spec is written to be executed directly. Feed it as the reference SRS/architecture; requirement IDs (e.g. PAY-01, TRU-02) and the tier/Trust Score tables anchor implementation prompts, tests, and PRs. The next step in your workflow is the queued prompt series that turns each module and phase into concrete, ordered build tasks against this stack and TMDS. End of document.

24. Local Development Environment & Tooling 24.1 Docker Compose (local Postgres + Redis) A docker-compose.yml at the repo root provisions local infrastructure for Docker Desktop: • postgres: postgis/postgis image (PostGIS preinstalled), port 5432, named volume for data, healthcheck (pg_isready), POSTGRES_DB=trustpro. • redis: redis:7-alpine, port 6379, appendonly persistence, healthcheck. Redis serves three roles in the architecture: BullMQ queues (notification dispatch + webhook retries), the rate-limit store, and the WebSocket pub/sub adapter for horizontal gateway scaling. 24.2 Root package.json scripts (canonical set) The root scripts are the single entry point for all workflows; contributors and CI never invoke app-level scripts directly. Canonical set (web-only, adapted from the proven template): • prepare: husky (pre-commit: lint-staged typecheck + lint on changed files). • env:sync: node scripts/sync-env.mjs, copies/derives per-app .env files from the root .env so secrets are defined once. • dev / dev:web / dev:api: env:sync first, then turbo run dev (parallel) or the single app; dev:api builds packages/database first. • build / build:web / build:api: turbo run build (with –filter per app). • lint / lint:i18n / type-check / test / format / full-check: turbo-run linting, an i18n ESLint pass that forbids hardcoded user-facing strings, workspace type-check, tests, prettier, and the combined pre-push gate. • db:up / db:down / db:reset: docker compose lifecycle; db:reset recreates containers then migrate:deploy + generate. • db:generate / db:push / db:studio / db:migrate / db:migrate:deploy / db:migrate:reset / db:seed: all delegate to packages/database. • dev:setup: one-shot bootstrap, build database package → db:up → generate → migrate → seed. • docker:up / docker:reset / docker:logs, backup:db (scripts/docker-db-backup.sh), keys (scripts/keygen.cjs for JWT/VAPID secrets), 32-bytes (openssl rand -base64 32), clean. 24.3 Environment management One root .env.example documents every variable (DATABASE_URL, DIRECT_URL, REDIS_URL, JWT secrets, VAPID keys, CAMPAY_, *S3*_, MAPBOX_TOKEN, CARTO_*, BIGDATACLOUD_KEY, RESEND_API_KEY). scripts/sync-env.mjs fans values out to apps; secrets never live in more than one file; nothing secret is committed.
25. CI/CD Workflows (GitHub Actions) Three workflows, adapted from the proven template: • deploy-production.yml: on push/PR to main. Job validate (pnpm install –frozen-lockfile → db:generate → type-check → lint); job db-migrate (main only, and only when the commit touches packages/database/prisma): prisma migrate deploy against DIRECT_URL, then a schema-drift gate: prisma migrate diff –from-url $DIRECT_URL –to-schema-datamodel schema.prisma –exit-code, failing the pipeline on drift; job summary writes a deployment summary (API auto-deployed by Railway branch integration; web by Vercel Git integration). • prisma-migrate.yml: path-triggered (packages/database/prisma/**) migrate-deploy + the same drift check, for schema-only pushes. • manual-seed.yml: workflow_dispatch gated by typing CONFIRM; runs db:generate then db:seed against production. Never runs automatically. Rules: migrations run in CI before servers boot, never at request time; migration_lock.toml committed (provider=postgresql); .sql files UTF-8 without BOM; secrets via repository secrets only.
26. Production Engineering Addendum (battle-tested lessons applied) These field-proven rules from the startup engineering guide are binding for TrustPro and referenced by the build prompts. 26.1 Notifications (dispatcher discipline) Covered in §17; in addition: seed default preferences at user creation; category names are a typed enum; store templateKey in logs, not rendered content; an admin dashboard reads NotificationLog for sent/failed/pending by channel, template, and date. 26.2 Frontend system rules • Theme: next-themes with a blocking flash-prevention script in the layout head (reads storage, applies class before paint); zero hydration flicker; toggle present in desktop nav and the mobile drawer; neither pure white nor pure black backgrounds in either mode. • Colour ratio 60/30/10: neutrals dominate (60%), Indigo primary supports (30%), Accent appears only on CTAs/hover/interactive highlights (10%) and must be visibly present on every primary CTA; documented in BRAND_GUIDELINES.md. • Motion: a central motion utility module (Anime.js v4) exports named presets (fadeUp/fadeDown/fadeLeft/fadeRight, scaleIn, blurReveal, stagger); transform/opacity only; 0.4-0.6s easeOut; every preset returns inert under prefers-reduced-motion; directional variation across landing sections; app/dashboard surfaces stay minimal. • Hover system: no static cards; standardized lift/shadow/scale/border-glow utilities; image containers overflow-hidden with slow scale; distinct CTA hover states. • Navigation: sticky header transparent→solid+blur after ~60px with a 200-300ms transition; animated mobile drawer (backdrop, focus trap, ESC, close on route change); nav links derive from a single page-sections constant (1:1 with real section ids, order matches the page) with IntersectionObserver active states; global [id]{scroll-margin-top} matching header height; hash clicks never trigger the route loader. • Layout groups: (marketing), (auth), (app), (admin); header+footer render on every customer-facing page including all auth screens; auth is split-screen on desktop (form left, brand panel right) and full-width on phones; only /admin has its own chrome. • Content-driven marketing components: FAQ and testimonials are database-driven with admin CRUD (never hardcoded arrays); FAQ uses a smooth animated accordion; testimonials render as an infinite marquee (pause on hover, drag, edge fade) seeded with at least 8 entries. • Ratings render decimals with half-stars (4.3 → 4 full + 1 half), aria-labelled, always computed server-side. • Filters: ranges (price, score) computed from data with sensible floors, never hardcoded; URL-synced state; branded empty states; skeletons during transitions. 26.3 Rendering & build resilience • Every generateStaticParams wraps its DB call in try/catch and returns [] on failure (warn, not error) so builds survive an unavailable database; audit next build output after every PR for accidental dynamic promotion of public pages. • Auth code never runs on Edge routes (Node runtime only). • Webhook payloads (Campay) are fully typed interfaces validated before processing, never any. 26.4 SEO hygiene • noindex by default: paginated variants beyond page 1, filtered catalog URLs, search results, account/auth/preview routes (meta + robots.txt). • Canonicals always server-derived and stripped of query params: filtered discovery → clean base; job/worker detail → clean slug; page 1 canonical for all pagination. • A route-metadata integrity check (CI or admin QA panel) asserts every public route exports title, description, canonical, and OG image. 26.5 Email & forms Branded transactional emails only (tokens, logo, header/footer, mobile-responsive), delivered through the channel service; form endpoints report delivery honestly ({ success, emailSent, message }, warning state when saved-but-not-sent); SMTP/provider timeouts configured; contact/public forms carry honeypots and rate limits. 26.6 Compliance & analytics baseline Legal pages (Terms, Privacy, Refund/Dispute policy) ship at launch as static pages; a lightweight analytics event layer tracks the conversion funnel (landing CTA → sign-up → verification steps → first job/quote → first escrow) with events named in one typed constant; social-preview (OG) images per key public route.

27. Version 3 changelog Web-only scope enforced end to end (native mobile removed); Redis + BullMQ layer added (queues, rate limiting, WS adapter); notifications rebuilt around the dispatcher pattern with NotificationLog, localized template store, multi-device web-push registry, E.164 phone normalisation, SMS deferred behind a feature flag; local dev environment defined (Docker Compose postgres+redis, canonical root scripts, env:sync, husky, keygen); CI/CD workflows specified (deploy-production with drift gate, path-triggered migrate, confirm-gated manual seed); production engineering addendum (§26) binding the startup-guide lessons; guest exploration view vs authenticated role-based homes confirmed (§3.1-3.2 access model).